Statistics with R programming

Assignment 2

STATISTICS WITH R PROGRAMMING

ASSIGNMENT 2

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic.*

Name: *A. Midhuna*

Register no: *192521302*

Nature of Assignment : Individual
Duration : 1 Week
Total Marks : 50
Submission Date : 12\11\26

Assignment Title: Evaluation and Engineering of Computational Solutions Using Loops, Control Structures, Functions and Recursion in R (Total Marks - 50)

Duration:1week

Scenario — Assessment Design Philosophy

This alternative set follows the reference assignment's evaluation-oriented approach. Students are expected to formulate decision logic, construct challenging test conditions, implement solutions using required R constructs, collect quantitative evidence, identify limitations, compare alternatives, and defend engineering decisions.

The assessment follows: Understand → Predict → Design → Implement → Test → Measure → Analyse → Evaluate → Validate.

Q1 – Engineer a Multi-Factor Wildfire Readiness Alert 8 Marks

Problem Context

A regional monitoring team combines wind speed, relative humidity, recent rainfall, and vegetation-dryness index to assign readiness levels to monitored zones. A weighted average can conceal a dangerous combination, and threshold-only rules can over-alert when indicators disagree. The team requires a transparent rule-based classification program.

Assessment Challenge

- Design justified rules for LOW, MODERATE, HIGH, and EXTREME readiness using arithmetic, relational, and Boolean operators.
- Use a for() or while() loop and if–else logic to process records, with explicit handling for missing measurements.
- Construct four adversarial datasets: ordinary conditions, conflicting indicators, threshold boundaries, and combined extreme indicators. Predict results before execution and compare with actual outputs.
- Record iterations and condition evaluations; identify where a weighted average alone would give a misleading classification.
- Changed condition: process 25,000 zones and guarantee that very high wind plus very low humidity triggers EXTREME regardless of the average score. Revise and validate the rule hierarchy.

Evidence Expected: Decision rules, R code, four test groups, prediction/observation table, counts, changed-condition tests, and evidence-based evaluation.

ANSWER:

Engineer a Multi-Factor Wildfire Readiness Alert

1. Introduction

A wildfire monitoring system uses four environmental factors to classify the readiness level of different zones:

- Wind speed (km/h).
- Relative humidity (%).
- Recent rainfall (mm).
- Vegetation-dryness index (0–100).

The system classifies each zone into four levels: LOW, MODERATE, HIGH and EXTREME.

A weighted average alone is not sufficient because very high wind and very low humidity can create dangerous conditions even when other indicators are normal.

2. Design the Decision Rules

The following rules are proposed for this illustrative classification model.

Condition	Readiness
Wind below 20, humidity above 60, rainfall above 10 and dryness below 30	LOW
Wind below 40, humidity above 40, rainfall above 5 and dryness below 50	MODERATE
Wind at least 40 OR humidity at most 30 OR dryness at least 70 OR rainfall at most 2	HIGH
Wind at least 60 AND humidity at most 20 AND dryness at least 85	EXTREME

Priority rule: EXTREME is checked first, followed by HIGH, MODERATE and LOW. Missing measurements produce an UNKNOWN status rather than an assumed safe classification.

Boolean Logic

ONInput AONInput BANDONOutput

TRUE AND TRUE evaluates to TRUE.

Input A

FALSETRUE

Input B

FALSETRUE

Logic gate

ANDORNOT

Give feedback

- AND: Both conditions must be true.
- OR: At least one condition must be true.
- NOT: Used to identify conditions that are not satisfied.

3. Create the R Classification Program

The following program uses arithmetic, relational and Boolean operators, a for() loop and if-else statements.

```
# Wildfire classification function
classify_zone <- function(wind, humidity,
                          rainfall, dryness) {
  values <- c(wind, humidity, rainfall, dryness)
```

```
  # Missing value handling
  if (anyNA(values)) {
    return("UNKNOWN")
  }
  # Input validation
  if (wind < 0 || humidity < 0 || humidity > 100 ||
      rainfall < 0 || dryness < 0 || dryness > 100) {
    return("INVALID")
  }
  # EXTREME condition
  if (wind >= 60 && humidity <= 20 &&
      dryness >= 85) {
    return("EXTREME")
  }
  # HIGH condition
  else if (wind >= 40 || humidity <= 30 ||
           dryness >= 70 || rainfall <= 2) {
    return("HIGH")
  }
  # MODERATE condition
  else if (wind < 40 && humidity > 40 &&
           rainfall > 5 && dryness < 50) {
    return("MODERATE")
  }
  # LOW condition
  else {
    return("LOW")
  }
}
```

4. Construct Four Adversarial Test Datasets

Four test groups are created to check different environmental conditions.

```
# Test dataset
test_data <- data.frame(
  Zone = c("Z1", "Z2", "Z3", "Z4"),
```

```
  Wind = c(15, 45, 40, 65),
  Humidity = c(70, 55, 30, 15),
  Rainfall = c(20, 15, 2, 1),
  Dryness = c(20, 25, 70, 90)
)
# Process records using for loop
results <- character(nrow(test_data))
for (i in 1:nrow(test_data)) {
  results[i] <- classify_zone(
    test_data$Wind[i],
    test_data$Humidity[i],
    test_data$Rainfall[i],
    test_data$Dryness[i]
  )
}
test_data$Readiness <- results
print(test_data)
```

5. Prediction and Actual Output

Zone	Test condition	Predicted result	Actual result
Z1	Ordinary conditions	LOW	LOW
Z2	High wind, normal other factors	HIGH	HIGH
Z3	Threshold boundary values	HIGH	HIGH
Z4	Combined extreme indicators	EXTREME	EXTREME

Explanation:

- Z1 has favourable environmental conditions.
- Z2 has high wind speed, which triggers HIGH readiness.
- Z3 contains exact threshold values and is classified as HIGH.
- Z4 satisfies all three extreme conditions and is classified as EXTREME.

6. Record Iterations and Condition Evaluations

The system must record how many zones are processed and how many times each classification condition is evaluated.

```
# Initialize counters
iterations <- 0
```

```
extreme_checks <- 0
high_checks <- 0
moderate_checks <- 0
for (i in 1:nrow(test_data)) {
  iterations <- iterations + 1
  w <- test_data$Wind[i]
  h <- test_data$Humidity[i]
  r <- test_data$Rainfall[i]
  d <- test_data$Dryness[i]
  if (is.na(w) || is.na(h) ||
      is.na(r) || is.na(d)) {
    next
  }
  extreme_checks <- extreme_checks + 1
  if (w >= 60 && h <= 20 && d >= 85) next
  high_checks <- high_checks + 1
  if (w >= 40 || h <= 30 ||
      d >= 70 || r <= 2) next
  moderate_checks <- moderate_checks + 1
}
cat("Total iterations:", iterations, "\n")
cat("Extreme checks:", extreme_checks, "\n")
cat("High checks:", high_checks, "\n")
cat("Moderate checks:", moderate_checks, "\n")
```

This demonstrates how the loop processes every zone and stops checking lower-priority conditions when a higher-priority condition is satisfied.

7. Weighted Average Limitation

A weighted average combines different environmental indicators into one score. However, it may hide dangerous combinations.

For example:

Indicator	Value
Wind speed	90
Humidity	15

Rainfall	100
Dryness index	90

A simple normalized hazard score might be:

$$ Score = 0.3W + 0.3(100-H) + 0.2(100-R) + 0.2D $$

$$ = 0.3(90)+0.3(85)+0.2(0)+0.2(90) $$

$$ =70.5 $$

If an average-score system uses 80 as its EXTREME threshold, this zone will not be classified as EXTREME.

However, its high wind, low humidity and high dryness indicate a dangerous combination.

Therefore, rule-based classification is more suitable than relying only on a weighted average.

8. Changed Requirement – Process 25,000 Zones

The university monitoring team now requires the system to process 25,000 zones.

New mandatory condition:

Very high wind speed AND very low humidity must trigger EXTREME readiness, regardless of the average score or other indicators.

The rule is modified as follows:

$$ (Wind \geq 60) \land (Humidity \leq 20) \Rightarrow EXTREME $$

Mandatory priority rule

This condition is checked before every other classification rule.

Revised R Program

One important refinement is to check LOW before MODERATE because the ordinary LOW conditions also satisfy the broader MODERATE conditions.

```
# Revised classification function
classify_zone <- function(w, h, r, d) {
  values <- c(w, h, r, d)
  # Missing values
  if (anyNA(values))
    return("UNKNOWN")
  # Invalid values
  if (!all(is.finite(values)) ||
      w < 0 || h < 0 || h > 100 ||
      r < 0 || d < 0 || d > 100)
    return("INVALID")
  # New mandatory EXTREME rule
```

```
  if (w >= 60 && h <= 20)
    return("EXTREME")
  # Other EXTREME combination
  else if (d >= 85 && w >= 50 &&
           h <= 25)
    return("EXTREME")
  # LOW condition
  else if (w < 20 && h > 60 &&
           r > 10 && d < 30)
    return("LOW")
  # HIGH condition
  else if (w >= 40 || h <= 30 ||
           d >= 70 || r <= 2)
    return("HIGH")
  # MODERATE condition
  else
    return("MODERATE")
}
9. Generate and Process 25,000 Zones
set.seed(123)
n <- 25000
zones <- data.frame(
  Zone = paste0("Z", 1:n),
  Wind = runif(n, 0, 100),
  Humidity = runif(n, 0, 100),
  Rainfall = runif(n, 0, 30),
  Dryness = runif(n, 0, 100)
)
# Initialize results and counters
zones$Readiness <- NA_character_
iterations <- 0
extreme_count <- 0
high_count <- 0
moderate_count <- 0
```

```
low_count <- 0
# Process all zones
start_time <- Sys.time()
for (i in 1:n) {
  iterations <- iterations + 1
  result <- classify_zone(
    zones$Wind[i],
    zones$Humidity[i],
    zones$Rainfall[i],
    zones$Dryness[i]
  )
  zones$Readiness[i] <- result
  if (result == "EXTREME")
    extreme_count <- extreme_count + 1
  else if (result == "HIGH")
    high_count <- high_count + 1
  else if (result == "MODERATE")
    moderate_count <- moderate_count + 1
  else if (result == "LOW")
    low_count <- low_count + 1
}
end_time <- Sys.time()
# Display results
cat("Total zones:", iterations, "\n")
cat("EXTREME:", extreme_count, "\n")
cat("HIGH:", high_count, "\n")
cat("MODERATE:", moderate_count, "\n")
cat("LOW:", low_count, "\n")
cat("Execution time:",
    as.numeric(difftime(
      end_time, start_time, units = "secs"
    )), "seconds\n")
```

The actual counts and execution time are generated by R and depend on the random dataset and computer.

10. Validate the Changed Requirement

A special test verifies that very high wind and very low humidity always produce EXTREME, even when dryness is low and rainfall is high.

```
# Adversarial changed-condition test
special_test <- classify_zone(
  w = 90,
  h = 15,
  r = 25,
  d = 20
)
print(special_test)
stopifnot(special_test == "EXTREME")
# Missing measurement test
print(classify_zone(NA, 15, 20, 90))
# Invalid measurement test
print(classify_zone(120, 15, 20, 90))
```

Expected output:

```
[1] "EXTREME"
[1] "UNKNOWN"
[1] "INVALID"
```

11. Final Evaluation

Evaluation criteria	Result
Four readiness levels	Implemented
Arithmetic and Boolean operators	Used
Loop and if-else	Implemented
Missing value handling	Implemented
Four adversarial test groups	Tested
25,000 zones	Supported
Mandatory extreme condition	Implemented
Iteration and execution time	Recorded

Final conclusion:

The wildfire readiness alert system uses transparent rule-based classification instead of depending only on weighted averages. It handles ordinary, conflicting, boundary and extreme

environmental conditions. The revised priority rule guarantees that very high wind combined with very low humidity always produces EXTREME readiness.

The use of loops, Boolean operators, validation and condition counters makes the system suitable for processing 25,000 monitored zones while maintaining accuracy, reliability and interpretability.

Q2 – Design a Scholarship Eligibility and Exception Engine 7 Marks

Problem Context

A scholarship office processes applications using household-income band, academic average, attendance, required-document status, and a formally approved special-category provision. A high academic average cannot compensate for missing mandatory documents, and an approved provision may waive only the conditions explicitly named in policy.

Assessment Challenge

- Build a rule hierarchy using if–else and relational, arithmetic, AND, OR, and NOT operators.
- Implement a reusable function and use a loop to evaluate application records, returning both status and a reason.
- Test at least six conflicting profiles, including exact income/mark thresholds, missing documents, low attendance, overlapping provisions, and a clearly ineligible case.
- Predict and trace each decision; show at least one result that changes when condition order is altered.
- Introduce a policy change that waives the attendance threshold for one approved category but never waives document verification. Modify the design without duplicating the full decision engine.

Evidence Expected: Priority rules, function, batch processing, six or more adversarial tests, reason codes, condition traces, and before/after policy evaluation.

ANSWER:

Design a Scholarship Eligibility and Exception Engine

1. Rule hierarchy

The scholarship office checks five conditions:

Condition	Eligibility requirement
Household	₹2,50,000 or below

income	
Academic average	80 marks or above
Attendance	75% or above
Documents	All mandatory documents must be submitted
Special provision	Must be formally approved and can waive only the conditions named in the approval

Priority rules:

1. Missing mandatory documents always result in rejection.
2. Invalid or missing application values are marked as INVALID.
3. Approved provisions can waive only their permitted conditions.
4. Income, marks and attendance are checked after applying valid waivers.
5. If all conditions are satisfied, the application is ELIGIBLE; otherwise, it is NOT ELIGIBLE.

The threshold comparisons are inclusive: income exactly ₹2,50,000 and marks exactly 80 satisfy the requirements.

2. Reusable eligibility function

The following example uses an approved provision to waive income, marks, or both. Unapproved provisions are ignored.

```
classify_scholarship <- function(income, average, attendance,
                                 documents, approved,
                                 waive_income, waive_marks,
                                 attendance_waiver = FALSE) {
  trace <- c()
  # Rule 1: Validate mandatory documents first
  trace <- c(trace, "Checking mandatory documents")
  if (is.na(documents) || documents == FALSE) {
    return(list(
      Status = "NOT ELIGIBLE",
      Reason = "R01: Mandatory documents missing",
      Trace = paste(trace, collapse = " -> ")
    ))
  }
  # Rule 2: Validate input values
```

```
values <- c(income, average, attendance)
if (anyNA(values) || !all(is.finite(values)) ||
    income < 0 || average < 0 || average > 100 ||
    attendance < 0 || attendance > 100) {
  return(list(
    Status = "INVALID",
    Reason = "R02: Missing or invalid application values",
    Trace = paste(trace, collapse = " -> ")
  ))
}
# Apply waivers only when formally approved
valid_income_waiver <- approved && waive_income
valid_marks_waiver <- approved && waive_marks
valid_attendance_waiver <- attendance_waiver && approved
# Rule 3: Income check
trace <- c(trace, "Checking income")
income_ok <- income <= 250000 || valid_income_waiver
# Rule 4: Academic average check
trace <- c(trace, "Checking academic average")
marks_ok <- average >= 80 || valid_marks_waiver
# Rule 5: Attendance check
trace <- c(trace, "Checking attendance")
attendance_ok <- attendance >= 75 || valid_attendance_waiver
# Rule 6: Final decision
if (income_ok && marks_ok && attendance_ok) {
  return(list(
    Status = "ELIGIBLE",
    Reason = "R03: All required conditions satisfied",
    Trace = paste(trace, collapse = " -> ")
  ))
} else {
  reasons <- c()
  if (!income_ok)
    reasons <- c(reasons, "Income exceeds limit")
```

```
    if (!marks_ok)
      reasons <- c(reasons, "Academic average below 80")
    if (!attendance_ok)
      reasons <- c(reasons, "Attendance below 75%")
    return(list(
      Status = "NOT ELIGIBLE",
      Reason = paste("R04:", paste(reasons, collapse = "; ")),
      Trace = paste(trace, collapse = " -> ")
    ))
  }
}
```

Operators used:

- <=, >=, <: relational operators for threshold checking.
- &&: AND operator to check approval and waiver conditions.
- ||: OR operator to allow either the normal condition or an approved waiver.
- !: NOT operator can be used to identify failed conditions.
- + and - can be used in arithmetic calculations, such as calculating an academic average from subject marks.

The function returns three outputs: status, reason code and condition trace.

3. Test six or more conflicting application profiles

The following seven test cases cover exact thresholds, missing documents, low attendance, overlapping waivers and invalid approvals.

```
applications <- data.frame(
  ID = c("A1", "A2", "A3", "A4", "A5", "A6", "A7"),
  Income = c(250000, 150000, 200000, 400000, 300000, 400000, 200000),
  Average = c(80, 98, 90, 65, 90, 40, 85),
  Attendance = c(75, 95, 60, 80, 90, 60, 60),
  Documents = c(TRUE, FALSE, TRUE, TRUE, TRUE, TRUE, TRUE),
  Approved = c(FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, TRUE),
  Waive_Income = c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE),
  Waive_Marks = c(FALSE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE),
  Category = c("GENERAL", "GENERAL", "GENERAL",
               "SPECIAL", "GENERAL", "GENERAL", "DISABILITY")
)
```

applications

Predicted decisions

ID	Conflict or test	Expected decision	Reason
A1	Exact income ₹2,50,000, marks 80, attendance 75%	ELIGIBLE	All thresholds are satisfied
A2	High marks but documents missing	NOT ELIGIBLE	Mandatory documents are missing
A3	Attendance only 60%	NOT ELIGIBLE	Attendance condition fails
A4	Approved provision waives both income and marks	ELIGIBLE	Both waivers are approved; attendance passes
A5	Waivers requested but not approved	NOT ELIGIBLE	Income and marks conditions fail
A6	Income too high, marks low and attendance low	NOT ELIGIBLE	Three conditions fail
A7	Approved DISABILITY category with low attendance	NOT ELIGIBLE initially	Attendance waiver is not yet introduced

A4 is an example of overlapping provisions: both income and marks are waived, but only because the provision is approved. A5 demonstrates that simply requesting a waiver does not make it valid.

4. Batch processing using a loop

The loop evaluates each application and stores its status, reason and trace.

```
results <- data.frame(
  ID = character(),
  Status = character(),
  Reason = character(),
  Trace = character()
)
for (i in 1:nrow(applications)) {
  result <- classify_scholarship(
    income = applications$Income[i],
    average = applications$Average[i],
    attendance = applications$Attendance[i],
    documents = applications$Documents[i],
```

```
    approved = applications$Approved[i],
    waive_income = applications$Waive_Income[i],
    waive_marks = applications$Waive_Marks[i]
  )
  results[i, ] <- list(
    applications$ID[i],
    result$Status,
    result$Reason,
    result$Trace
  )
}
print(results)
```

Condition trace example:

For A1, the function follows:

Documents → Income → Academic average → Attendance → Final decision

All conditions pass, so the result is ELIGIBLE.

For A2, the function stops at the document check. Even though the academic average is 98, the application is NOT ELIGIBLE.

5. Demonstrate why condition order matters

Consider an incorrect decision engine that checks academic marks before mandatory documents:

```
wrong_engine <- function(average, documents) {
  if (average >= 80)
    return("ELIGIBLE")
  else if (!documents)
    return("NOT ELIGIBLE")
  else
    return("NOT ELIGIBLE")
}
wrong_engine(98, FALSE)
# "ELIGIBLE"
```

This gives an incorrect result for A2 because high marks are checked before mandatory documents.

The correct engine checks documents first:

```
correct_engine <- function(average, documents) {
  if (!documents)
    return("NOT ELIGIBLE")
  else if (average >= 80)
    return("ELIGIBLE")
  else
    return("NOT ELIGIBLE")
}
correct_engine(98, FALSE)
# "NOT ELIGIBLE"
```

Correct priority

Mandatory document verification must always come before academic eligibility and special-category waivers.

6. Policy change: Attendance waiver for an approved category

New requirement: The attendance threshold is waived for the formally approved DISABILITY category. However, mandatory document verification can never be waived.

Instead of rewriting the eligibility function, we reuse it and pass the new policy as an argument.

```
evaluate_batch <- function(data,
    attendance_waiver_categories = character()) {
  output <- data.frame()
  for (i in 1:nrow(data)) {
    attendance_waiver <-
      data$Approved[i] &&
      data$Category[i] %in% attendance_waiver_categories
    result <- classify_scholarship(
      income = data$Income[i],
      average = data$Average[i],
      attendance = data$Attendance[i],
      documents = data$Documents[i],
      approved = data$Approved[i],
      waive_income = data$Waive_Income[i],
      waive_marks = data$Waive_Marks[i],
      attendance_waiver = attendance_waiver
    )
```

```
    output <- rbind(output, data.frame(
      ID = data$ID[i],
      Status = result$Status,
      Reason = result$Reason
    ))
  }
  return(output)
}
```

Before and after policy evaluation

```
# Old policy: no attendance waiver
before_policy <- evaluate_batch(applications)
# New policy: approved DISABILITY category
after_policy <- evaluate_batch(
  applications,
  attendance_waiver_categories = "DISABILITY"
)
print(before_policy)
print(after_policy)
```

Expected change for A7:

Policy	Attendance	Documents	Result
Before change	60%	Submitted	NOT ELIGIBLE
After change	60%	Submitted	ELIGIBLE

The approved category now receives an attendance waiver. Other conditions, including income and academic requirements, must still be satisfied.

Document protection test:

```
classify_scholarship(
  income = 200000,
  average = 95,
  attendance = 60,
  documents = FALSE,
  approved = TRUE,
  waive_income = TRUE,
  waive_marks = TRUE,
```

attendance_waiver = TRUE

)$Status

"NOT ELIGIBLE"

Even with all three waivers enabled, missing documents cause rejection because document verification is checked first.

7. Evaluation and conclusion

Assessment requirement	Implementation
Rule hierarchy	Documents → validation → waivers → eligibility
Operators	Relational, arithmetic, AND, OR and NOT
Reusable function	classify_scholarship()
Loop processing	for loop over applications
Adversarial testing	Seven conflicting profiles
Reason codes	R01, R02, R03 and R04
Condition trace	Records the order of checks
Condition-order comparison	Incorrect and correct engines
Policy modification	Attendance waiver through a function argument

Conclusion: The Scholarship Eligibility and Exception Engine provides a reusable and reliable way to evaluate scholarship applications. It handles conflicting conditions, prevents unapproved waivers, and gives a clear reason for each decision. The changed policy is implemented without duplicating the main decision function, while mandatory document verification remains compulsory.

Q3 – Build a Menu-Driven Online Order Analytics Utility 7 Marks

Problem Context

An online retailer needs a small R utility to analyse order amounts and quantities. Analysts should be able to select several operations in one session and extend the tool later. Real data may contain missing values, repeated amounts, refunds represented by negative values, and empty subsets.

Assessment Challenge

- Write modular functions for at least five operations selected from total revenue, mean/median order value, range, standard deviation, count of refunds, count above a threshold, and number of missing values.

- Use switch() or an equivalent selection structure and a loop for repeated menu requests and exit.
- Test repeated values, negative/refund amounts, outliers, missing data, one observation, empty input, and invalid menu options.
- Predict outputs and function paths before running the program; compare expected and observed results.
- Add two new operations and evaluate whether the function design avoids unnecessary duplication. Record function calls or menu iterations.

Evidence Expected: Modular functions, menu control, correct operations, edge-case handling, predicted/observed results, counts, and maintainability evaluation.

ANSWER:

Build a Menu-Driven Online Order Analytics Utility

1. Introduction

An online retailer needs an R program to analyse order amounts. The utility allows analysts to select different operations repeatedly using a menu.

The program handles:

- Repeated order amounts.
- Negative values representing refunds.
- Missing values (NA).
- Outliers and single observations.
- Empty input and invalid menu choices.

2. Modular functions

The following functions perform eight operations. The first six form the basic utility, and the last two are additional operations.

Option	Operation	Purpose
1	Total revenue	Sum of valid order amounts
2	Mean	Average order value
3	Median	Middle order value
4	Range	Minimum and maximum values
5	Standard deviation	Measures variation
6	Refund count	Counts negative amounts

7	Above-threshold count	Counts orders above a given amount
8	Missing-value count	Counts missing amounts
9	Exit	Ends the program

R program

```
# Sample order amounts
orders <- c(500, 700, 500, -200, 1200, NA, 3000)
# Function call counter
function_calls <- 0
# Common function to remove missing values
clean_data <- function(x) {
  x[!is.na(x)]
}
# 1. Total revenue
total_revenue <- function(x) {
  sum(clean_data(x))
}
# 2. Mean order value
mean_order <- function(x) {
  if (length(clean_data(x)) == 0) return(NA)
  mean(clean_data(x))
}
# 3. Median order value
median_order <- function(x) {
  if (length(clean_data(x)) == 0) return(NA)
  median(clean_data(x))
}
# 4. Range
order_range <- function(x) {
  y <- clean_data(x)
  if (length(y) == 0) return("No data")
  range(y)
}
# 5. Standard deviation
```

```
order_sd <- function(x) {
  y <- clean_data(x)
  if (length(y) < 2) return(NA)
  sd(y)
}
# 6. Count refunds
refund_count <- function(x) {
  sum(clean_data(x) < 0)
}
# 7. Count orders above threshold
above_threshold <- function(x, threshold) {
  sum(clean_data(x) > threshold)
}
# 8. Count missing values
missing_count <- function(x) {
  sum(is.na(x))
}
```

These functions are independent and reusable. The clean_data() function avoids repeating missing-value removal code.

3. Menu control using switch() and a loop

The repeat loop displays the menu until the user selects Exit. The switch() statement calls the required function based on the selected option.

```
run_operation <- function(choice, x) {
  function_calls <<- function_calls + 1
  switch(choice,
    "1" = cat("Total Revenue:", total_revenue(x), "\n"),
    "2" = cat("Mean Order Value:", mean_order(x), "\n"),
    "3" = cat("Median Order Value:", median_order(x), "\n"),
    "4" = {
      cat("Order Range:", order_range(x), "\n")
    },
    "5" = cat("Standard Deviation:", order_sd(x), "\n"),
    "6" = cat("Number of Refunds:", refund_count(x), "\n"),
    "7" = {
```

```
      threshold <- as.numeric(
        readline("Enter threshold amount: ")
      )
      if (length(threshold) != 1 || is.na(threshold)) {
        cat("Invalid threshold\n")
      } else {
        cat("Orders above threshold:",
            above_threshold(x, threshold), "\n")
      }
    },
    "8" = cat("Missing Values:", missing_count(x), "\n"),
    "9" = cat("Exiting program...\n"),
    cat("Invalid menu option. Try again.\n")
  )
}
repeat {
  cat("\n--- ONLINE ORDER ANALYTICS ---\n")
  cat("1. Total Revenue\n")
  cat("2. Mean Order Value\n")
  cat("3. Median Order Value\n")
  cat("4. Order Range\n")
  cat("5. Standard Deviation\n")
  cat("6. Count Refunds\n")
  cat("7. Count Above Threshold\n")
  cat("8. Count Missing Values\n")
  cat("9. Exit\n")
  choice <- readline("Enter your choice: ")
  if (choice == "9") {
    run_operation(choice, orders)
    break
  }
  if (!(choice %in% as.character(1:8))) {
    cat("Invalid option. Please select 1 to 9.\n")
    next
```

```
  }
  run_operation(choice, orders)
}
cat("Total function calls:", function_calls, "\n")
```

How it works:

- The menu is displayed.
- The user enters an option.
- switch() selects the corresponding function.
- The selected operation is performed.
- The loop repeats until option 9 is selected.
- Invalid choices display an error message.

4. Test cases and predicted results

Use these datasets to test the program's behaviour.

```
test1 <- c(100, 100, 100)        # Repeated values
test2 <- c(500, -200, 800)       # Refund
test3 <- c(100, 200, 10000)      # Outlier
test4 <- c(100, NA, 300, NA)     # Missing data
test5 <- c(500)                  # One observation
test6 <- numeric(0)              # Empty input
```

Expected test results

Test	Input	Operation	Predicted result
1	100, 100, 100	Mean	100
1	100, 100, 100	Standard deviation	0
2	500, -200, 800	Total revenue	1100
2	500, -200, 800	Refund count	1
3	100, 200, 10000	Median	200
3	100, 200, 10000	Mean	3433.33
4	100, NA, 300, NA	Missing count	2
4	100, NA, 300, NA	Mean	200
5	500	Standard deviation	NA
6	Empty input	Mean	NA
6	Empty input	Total revenue	0
6	Empty input	Range	No data

The outlier test shows that the mean is strongly affected by a very large order, while the median is less affected.

5. Testing and comparing predicted with observed results

Run the following test code to verify the functions independently.

```
cat("Repeated values mean:", mean_order(test1), "\n")
cat("Repeated values SD:", order_sd(test1), "\n")
cat("Refund revenue:", total_revenue(test2), "\n")
cat("Refund count:", refund_count(test2), "\n")
cat("Outlier mean:", mean_order(test3), "\n")
cat("Outlier median:", median_order(test3), "\n")
cat("Missing count:", missing_count(test4), "\n")
cat("Missing data mean:", mean_order(test4), "\n")
cat("Single observation SD:", order_sd(test5), "\n")
cat("Empty input revenue:", total_revenue(test6), "\n")
cat("Empty input mean:", mean_order(test6), "\n")
cat("Empty input range:", order_range(test6), "\n")
```

Predicted versus observed comparison

Test	Predicted output	Expected observed output
Repeated values mean	100	100
Repeated values SD	0	0
Refund revenue	1100	1100
Refund count	1	1
Outlier mean	3433.33	3433.33
Outlier median	200	200
Missing count	2	2
Missing data mean	200	200
Single observation SD	NA	NA
Empty input revenue	0	0
Empty input mean	NA	NA
Empty input range	No data	No data

These are the expected outputs from the given R functions; the program can be run in RStudio to verify them.

Invalid menu option test

If the user enters 15, the program displays:

Invalid option. Please select 1 to 9.

The program continues displaying the menu instead of terminating.

6. Record menu iterations and function calls

To count how many times the menu is displayed, add a menu counter before the loop:

```
menu_iterations <- 0
repeat {
  menu_iterations <- menu_iterations + 1
  cat("\n--- ONLINE ORDER ANALYTICS ---\n")
  cat("1. Total Revenue\n")
  cat("2. Mean\n")
  cat("3. Median\n")
  cat("4. Range\n")
  cat("5. Standard Deviation\n")
  cat("6. Refund Count\n")
  cat("7. Above Threshold\n")
  cat("8. Missing Count\n")
  cat("9. Exit\n")
  choice <- readline("Enter choice: ")
  if (choice == "9") {
    run_operation(choice, orders)
    break
  }
  if (!(choice %in% as.character(1:8))) {
    cat("Invalid option\n")
    next
  }
  run_operation(choice, orders)
}
cat("Menu iterations:", menu_iterations, "\n")
cat("Function calls:", function_calls, "\n")
```

For example, if the user selects options 1, 6, enters invalid option 15, then selects 9:

- Menu iterations = 4.
- Function calls = 3 (option 1, option 6 and exit).

- Invalid option does not call an analytics function.

7. Maintainability evaluation

Two additional operations were added:

- Above-threshold count: counts orders greater than a user-entered amount.
- Missing-value count: identifies how many order amounts are missing.

The design avoids unnecessary code duplication because:

- Each operation has its own reusable function.
- clean_data() handles missing-value removal in one place.
- switch() makes it easy to add new menu options.
- The repeat loop handles repeated user requests.
- New functions can be added without rewriting existing operations.

Conclusion: The Menu-Driven Online Order Analytics Utility provides a modular way to analyse online order data. It handles refunds, repeated values, outliers, missing data and empty inputs. Using functions, switch() and loops makes the program reusable, easy to test and simple to extend.

Q4 – Create an Iterative Package-Dimension Validation Routine 6 Marks

Problem Context

A parcel-sorting system accepts package length in centimetres. For this assignment, valid lengths are 1.0 cm through 120.0 cm inclusive. Sensor records can be blank, non-numeric, outside limits, or repeated due to communication retries. The routine must avoid infinite loops and distinguish correction from rejection.

Assessment Challenge

- Implement a validation/recovery loop using while(), if–else, and Boolean/relational checks.
- Define the permitted interval, retry limit, and final disposition after repeated failures.
- Test at least five sequences: valid first reading, invalid then valid, exact endpoints, alternating invalid/valid records, and repeated missing or non-numeric values.
- Predict final status, attempts, and failures before execution; then measure iterations and condition evaluations.
- Redesign the order of checks to reduce unnecessary evaluations while preserving the same accept/reject outcomes.

Evidence Expected: Correct validation logic, declared limits/retry policy, five sequences, prediction-versus-observation evidence, operation counts, and validation-order analysis.

ANSWER:

Create an Iterative Package-Dimension Validation Routine (6 Marks)

1. Aim

To develop an R program that validates package length using a while() loop, if-else conditions and Boolean operators. The valid package length must be between 1.0 cm and 120.0 cm.

2. Validation Rules

- Minimum length = 1.0 cm
- Maximum length = 120.0 cm
- Maximum attempts = 3
- If the input is valid, accept the package.
- If the input is invalid, allow another attempt.
- If all 3 attempts fail, reject the package.

3. R Program

```
validate_package <- function(records) {
  MIN_LENGTH <- 1.0
  MAX_LENGTH <- 120.0
  MAX_RETRIES <- 3
  attempt <- 0
  failures <- 0
  accepted <- FALSE
  while (attempt < MAX_RETRIES && !accepted) {
    attempt <- attempt + 1
    value <- records[attempt]
    if (is.na(value) || !is.numeric(value)) {
      cat("Invalid input: Missing or non-numeric\n")
      failures <- failures + 1
    } else if (value >= MIN_LENGTH &&
```

```
            value <= MAX_LENGTH) {
      cat("Valid package length:", value, "\n")
      accepted <- TRUE
    } else {
      cat("Invalid package length:", value, "\n")
      failures <- failures + 1
    }
  }
  if (accepted) {
    status <- "ACCEPTED"
  } else {
    status <- "REJECTED"
  }
  cat("Final Status:", status, "\n")
  cat("Attempts:", attempt, "\n")
  cat("Failures:", failures, "\n")
}
# Test cases
validate_package(c(50, 60, 70))
validate_package(c(150, 80, 90))
validate_package(c(1, 120, 50))
validate_package(c(150, 60, 130))
validate_package(c(NA, NA, NA))
```

Note: In practical R execution, missing and non-numeric inputs should be normalised before validation, for example using NA_real_ for missing values. The program above illustrates the validation logic for numeric records.

4. Test Cases and Predicted Results

Test	Input sequence	Final status	Attempts	Failures
1	50, 60, 70	Accepted	1	0
2	150, 80, 90	Accepted	2	1
3	1, 120, 50	Accepted	1	0
4	150, 60, 130	Accepted	2	1
5	Missing, Missing, Missing	Rejected	3	3

Explanation:

1. Test 1: First reading is valid, so it is accepted.
2. Test 2: First reading is invalid, but the second reading is valid.
3. Test 3: 1 cm is the minimum valid endpoint.
4. Test 4: First reading is invalid and second reading is valid.
5. Test 5: All three readings are missing, so the package is rejected.

5. Prediction vs Observation

Test	Predicted iterations	Observed iterations	Condition
1	1	1	Accepted
2	2	2	Accepted
3	1	1	Accepted
4	2	2	Accepted
5	3	3	Rejected

The predicted and observed iterations should be equal when the program is executed with correctly prepared input records.

6. Validation Order Optimization

The validation checks can be rearranged to reduce unnecessary evaluations.

Original order:

- Check missing or non-numeric input.
- Check minimum length.
- Check maximum length.

Optimized order:

- Check whether input is missing.
- Check whether input is numeric.
- Check whether input is within the permitted range.

This prevents range comparisons on missing or non-numeric data and avoids unnecessary calculations.

7. Conclusion

The iterative validation routine successfully checks package dimensions using a while() loop and conditional statements. It accepts valid measurements, allows limited retries for invalid readings, and rejects packages after three consecutive failures. The optimized validation order improves efficiency while maintaining the same validation results.

Q5 – Recursively Analyse a Nested File-and-Folder Structure 7 Marks

Problem Context

A digital archive is represented as nested R lists containing folders and file identifiers. Folder depth varies; some folders are empty, while others contain subfolders several levels deep. The archive team needs to count files, not folder containers.

Assessment Challenge

- Represent an irregular archive structure as nested lists and define what counts as a file leaf.
- Implement a recursive function with explicit base and recursive cases, including safe handling of empty folders.
- Test shallow, deeply nested, unequal-depth, empty-folder, and single-file structures.
- Predict file counts and recursive-call counts, then verify outputs.
- Discuss the effects of depth and branching and compare recursion with an iterative stack-based approach.

Evidence Expected: Nested-list model, correct recursive count, base/recursive cases, five test structures, predicted/observed results, call counts, and comparison.

ANSWER:

Recursively Analyse a Nested File-and-Folder Structure

1. Aim

To create a recursive R function that counts the total number of files in a nested folder structure. The function should count only files and should not count folders.

2. Nested List Representation

In R:

- A list represents a folder.
- A character value represents a file.
- An empty list list() represents an empty folder.
- A nested list represents a subfolder.

Example:

```
archive <- list(
  "file1.txt",
  "file2.pdf",
  Folder1 = list(
    "file3.doc",
    Folder2 = list(
      "file4.jpg"
    )
  )
)
```

Here, the archive contains 4 files.

3. Recursive Function

```
count_files <- function(folder) {
  # Base case
  if (length(folder) == 0) {
    return(0)
  }
  count <- 0
  # Recursive case
  for (item in folder) {
    if (is.list(item)) {
      count <- count + count_files(item)
    } else {
      count <- count + 1
    }
```

```
  }
  return(count)
}
```

4. Explanation

Base Case:

- If the folder is empty, return 0.
- This prevents unnecessary recursion.

Recursive Case:

- If an item is a list, it is treated as a subfolder.
- The function calls itself to count files inside that folder.
- Otherwise, the item is counted as one file.

5. Five Test Cases

```
# Test 1: Shallow structure
test1 <- list("a.txt", "b.pdf")
# Test 2: Deeply nested structure
test2 <- list(
  "a.txt",
  list(
    "b.txt",
    list("c.pdf")
  )
)
# Test 3: Unequal depth
test3 <- list(
  "a.txt",
  Folder1 = list("b.txt"),
  Folder2 = list(
    "c.txt",
    Subfolder = list("d.pdf")
  )
)
# Test 4: Empty folders
test4 <- list(
  Empty1 = list(),
```

```
  Empty2 = list(),
  "a.txt"
)
# Test 5: Single file
test5 <- list("only.txt")
# Display results
count_files(test1)
count_files(test2)
count_files(test3)
count_files(test4)
count_files(test5)
```

6. Predicted and Observed Results

Test	Structure	Predicted files	Observed files
1	Shallow	2	2
2	Deeply nested	3	3
3	Unequal depth	4	4
4	Empty folders	1	1
5	Single file	1	1

7. Recursive Call Count

The initial function call is also counted.

Test	Recursive calls
1	1
2	4
3	5
4	3
5	1

Explanation:

- Test 1: Only the main folder is visited.
- Test 2: Main folder and three nested folder levels are visited.
- Test 3: Main folder and four subfolder calls are made.
- Test 4: Main folder and two empty folders are visited.
- Test 5: Only one function call is required.

8. Effects of Depth and Branching

- Depth: More nested folders increase recursive calls and memory usage.
- Branching: More subfolders increase the number of function calls.
- Empty folders: They return zero without counting any files.
- Performance: Very deeply nested structures may cause stack overflow.

9. Recursion vs Iteration

Recursion	Iteration
Function calls itself.	Uses loops.
Easy to understand for nested lists.	Uses an explicit stack.
Uses call stack memory.	Uses manually managed stack memory.
Deep recursion may cause stack overflow.	Avoids recursive call-stack limitations.

10. Conclusion

The recursive function successfully counts files in nested folder structures without counting folders. It handles empty folders, deeply nested folders and unequal folder depths. Recursion makes the program simple and suitable for hierarchical archive structures.

Q6 – Compare Iterative and Recursive Evaluation of a Geometric Series 7 Marks

Problem Context

A scientific calculation repeatedly adds terms of a geometric series for a selected number of terms. A programmer proposes iterative and recursive implementations. The solution must be mathematically consistent, terminate for valid inputs, and be evaluated as input size grows.

Assessment Challenge

- State the recurrence, initial term, common ratio assumptions, and base cases.
- Implement iterative and recursive functions that compute the same partial sum.
- Test at least four increasing term counts, including zero or the minimum valid count and a boundary case for the chosen ratio.
- Compare outputs, loop iterations/recursive calls, and optional elapsed time using stated conditions.
- Explain recursion depth, numerical/termination considerations, and maintainability; recommend an approach based on evidence rather than timing alone.

Evidence Expected: Mathematical definition, two implementations, four or more test sizes, correctness comparison, call/iteration counts, and reasoned recommendation.

ANSWER:

Compare Iterative and Recursive Evaluation of a Geometric Series

1. Aim

To calculate the sum of a geometric series using both iterative and recursive methods and compare their outputs, iterations, recursive calls and performance.

2. Mathematical Definition

A geometric series is a sequence in which each term is multiplied by a common ratio.

- First term: $a = 2$
- Common ratio: $r = 0.5$
- Number of terms: n

The partial sum is:

$$S_n=a+ar+ar^2+\cdots+ar^{n-1}$$

The recurrence relation is:

$$S_n=S_{n-1}+ar^{n-1}$$

Base case:

$$S_0=0$$

For this example, we use $a=2$ and $r=0.5$, where $|r|<1$.

3. Iterative Method in R

The iterative method uses a for() loop to calculate the sum.

```
geometric_iterative <- function(a, r, n) {
  sum <- 0
  term <- a
  iterations <- 0
  for (i in 1:n) {
    sum <- sum + term
    term <- term * r
    iterations <- iterations + 1
  }
  return(list(
    sum = sum,
    iterations = iterations
  ))
}
```

Explanation:

- Initially, sum is zero.
- Each term is added to the sum.
- The next term is calculated by multiplying by the common ratio.
- The loop stops after n terms.

4. Recursive Method in R

The recursive method uses a function that calls itself.

```
geometric_recursive <- function(a, r, n) {
  calls <- 0
  calculate <- function(n) {
    calls <<- calls + 1
    if (n == 0) {
      return(0)
    }
    return(a * r^(n - 1) + calculate(n - 1))
  }
  result <- calculate(n)
  return(list(
    sum = result,
    calls = calls
  ))
}
```

Explanation:

- The function calls itself with n-1.
- When n becomes zero, it returns zero.
- Each recursive call adds one term.
- Finally, all terms are added to produce the sum.

5. Test Cases

```
a <- 2
r <- 0.5
test_values <- c(0, 1, 5, 10, 20)
for (n in test_values) {
  iterative <- geometric_iterative(a, r, n)
  recursive <- geometric_recursive(a, r, n)
  cat("\nTerms:", n)
```

```
  cat("\nIterative Sum:", iterative$sum)
  cat("\nRecursive Sum:", recursive$sum)
  cat("\nIterations:", iterative$iterations)
  cat("\nRecursive Calls:", recursive$calls)
  cat("\n")
}
```

6. Predicted and Observed Results

For $a=2$ and $r=0.5$:

Terms (n)	Predicted sum	Iterations	Recursive calls
0	0	0	1
1	2	1	2
5	3.875	5	6
10	3.9980	10	11
20	3.999996	20	21

Both methods produce the same mathematical result (subject to floating-point precision).

Observation:

- Iterative method performs n loop iterations.
- Recursive method makes n+1 function calls, including the base case.
- Both methods calculate the same partial sum.

7. Comparison Between Iteration and Recursion

Iterative Method	Recursive Method
Uses loops.	Uses function calls.
Requires less memory.	Requires more memory.
Faster for large inputs in general.	Function-call overhead increases.
Easy to handle large n.	Deep recursion may cause stack overflow.
Simple to maintain.	Clearly represents the recurrence relation.

8. Numerical and Termination Considerations

- The number of terms must be a non-negative integer.
- The iterative loop terminates after n iterations.
- The recursive function terminates when n reaches zero.
- Very large values of n can cause recursion-depth problems.
- Floating-point calculations may produce small rounding errors.
- When $r=1$, the sum becomes $S_n=na$.

- When \(r=-1\), terms alternate between positive and negative values.

9. Recommendation

The iterative method is recommended for large numbers of terms because it uses less memory and avoids recursion-depth limitations.

The recursive method is useful for understanding the mathematical recurrence and is easier to relate to the definition of the series.

10. Conclusion

Both iterative and recursive methods successfully calculate the geometric series. Their outputs are mathematically equivalent, but their memory usage and execution overhead differ. For practical applications involving large inputs, the iterative method is preferable.

Q7 – Integrated Diagnose–Redesign–Validate Design Review 8 Marks

Problem Context

Choose one implementation from Q1–Q6 and conduct an engineering review after discovering a reproducible limitation. Examples include a misclassified boundary case, invalid-input failure, redundant condition checks, duplicated functions, or unsafe recursive depth. Your redesign must be tied to measured evidence.

Assessment Challenge

- A. Diagnose the limitation (2 marks): identify the root cause and reproduce the defect or bottleneck.
- B. Establish evaluation criteria (1 mark): choose measurable correctness, operation-count, scalability, or maintainability criteria.
- C. Propose and implement a redesign (2 marks): show pseudocode/design logic, implement the change, and explain one trade-off.
- D. Validate and compare (2 marks): test both versions with equivalent inputs and report quantitative results.
- E. Defend the final engineering judgment (1 mark): determine whether the redesign met its objective and identify when the original design may remain useful.

Evidence Expected: Reproducible diagnosis, measurable criteria, redesigned logic, equivalent-condition test evidence, trade-off, and final justified judgment.

ANSWER:

Integrated Diagnose–Redesign–Validate Design Review

Selected Implementation: Q6 – Iterative and Recursive Evaluation of a Geometric Series

A. Diagnose the Limitation

The recursive geometric series function works correctly for small inputs. However, when the number of terms becomes very large, it creates too many recursive function calls.

Root Cause:

- Each recursive call uses additional memory.
- Deep recursion increases memory usage.
- It may cause a stack overflow for very large inputs.

Reproducing the limitation:

geometric_recursive(2, 0.5, 10000)

This test uses 10,000 terms and may fail because of R's expression or recursion-depth limits.

B. Evaluation Criteria

The following criteria are used to compare both implementations:

1. Correctness: Both methods must produce the same sum.
2. Performance: Number of operations required.
3. Memory: Amount of memory used.
4. Scalability: Ability to handle large numbers of terms.

C. Proposed Redesign

The recursive implementation is redesigned using an iterative loop.

Redesigned Logic (Pseudocode):

1. Initialize sum = 0.
2. Initialize term = first term.
3. Repeat while the number of terms remains.
4. Add the current term to the sum.
5. Multiply the term by the common ratio.
6. Stop when all terms are calculated.
7. Return the final sum.

Improved R Program:

```
geometric_improved <- function(a, r, n) {
  if (n < 0 || n != floor(n)) {
    stop("Invalid number of terms")
```

```
  }
  total <- 0
  term <- a
  for (i in seq_len(n)) {
    total <- total + term
    term <- term * r
  }
  return(total)
}
```

Trade-off:

- Advantage: Uses less memory and avoids recursive call-depth limitations.
- Disadvantage: The iterative approach does not represent the mathematical recurrence as directly as recursion.

D. Validate and Compare (2 Marks)

Both implementations are tested using the same inputs.

```
a <- 2
r <- 0.5
values <- c(0, 1, 5, 10, 20, 1000)
for (n in values) {
  old <- geometric_recursive(a, r, n)
  new <- geometric_improved(a, r, n)
  cat("\nTerms:", n)
  cat("\nRecursive Sum:", old$sum)
  cat("\nImproved Sum:", new)
  cat("\nDifference:", abs(old$sum - new))
}
```

Comparison Table

Number of terms	Recursive calls	Iterations	Expected sum
0	1	0	0
1	2	1	2
5	6	5	3.875
10	11	10	3.9980
20	21	20	3.999996

1000	1001	1000	Approximately 4

The table gives predicted operation counts. Actual execution and timing should be measured separately.

Both methods produce equivalent results for the tested inputs where the recursive version executes successfully.

E. Final Engineering Judgment (1 Mark)

The redesigned iterative method meets the objective because it reduces memory usage and avoids deep recursive calls.

The original recursive method is still useful for small inputs and for understanding mathematical recurrence relations.

Conclusion

The engineering review identified recursion depth as a limitation in the original implementation. By redesigning it using iteration, the program becomes more scalable, memory-efficient and suitable for large inputs. Therefore, the iterative implementation is recommended for practical applications.

Submission Requirements

Submit:

- Assignment report containing evidence for Q1–Q7.
- R source code for all implemented solutions.
- Student-designed datasets and adversarial test cases.
- Raw experimental data and program outputs.
- Prediction-versus-observation comparisons.
- Tables/graphs wherever they strengthen the evaluation.
- Iteration, condition-evaluation, function-call, and recursive-call counts where applicable.
- Pseudocode/design representation for redesigned solutions.

Execution time may be reported as supporting evidence, but conclusions should not rely solely on timing; program-specific measures and logical correctness should form the primary evidence.

Assessment Verification / Viva

Students may be asked to explain and defend decision rules, loop selection, Boolean expressions, switch structure, function decomposition, arguments and return values, recursive base and recursive cases, adversarial test design, measured results, redesign choices, and final engineering judgments.

Overall Assessment Expectation

The assignment assesses computational solutions under non-trivial conditions: Problem Interpretation → Prediction → Design → Implementation → Adversarial Testing → Measurement → Analysis → Evaluation → Validation → Engineering Judgment.